UK Biobank cholesterol application

fine-mapping-benchmark, branch application/ukb-cholesterol

generated 3 November 2026 from application/README.md at 1aa0fd7

Contents

UK Biobank cholesterol application **2**

1. Data . . . 2
    1.1 GWAS summary statistics . . . 2
    1.2 LD . . . 3
    1.3 Loci . . . 3
    1.4 Annotations . . . 3
    1.5 PolyFun prior — computed here, not downloaded . . . 3
    The PolyFun annotations are binned, and why . . . 4
    The analysed block set: 254 of 265 . . . 5
    Half the per-SNP heritabilities come out negative . . . 5
    1.6 Software (all upstream, unmodified) . . . 7
2. Preprocessing, exactly as performed . . . 7
    Step A — score every LDetect block (`00a_score_blocks.py`) . . . 7
    Step B — select loci and assign LD windows (`00_make_blocks.py`) . . . 7
    Step C — build per-block inputs (`prep_block.py`) . . . 8
    Resulting scale . . . 8
3. Running . . . 8
    Bootstrap once . . . 8
    Where to run things . . . 9
    Submit . . . 9
    Check before committing CPU . . . 10
    Collect . . . 10
4. Diagnostics . . . 10
5. Resources . . . 11
    Verified end to end . . . 12
    The LD regularisation defect (fixed) . . . 12
    Building the PolyFun environment . . . 13
6. Results . . . 13

6.1 What the seven columns are . . . 14
6.2 Headline, signal regions only . . . 15
6.3 Do the confident calls carry marginal evidence? . . . 15
6.4 Localisation: what |Z| does each method's own top variant carry? . . . 16
6.5 Two different fixes, two different effects . . . 16
6.6 Saturation is gone, and it is measurable . . . 17
6.7 Upstream BEATRICE emits almost no credible sets . . . 17
6.8 Where the methods disagree . . . 17
6.9 LD quality . . . 17
6.10 Which annotations the shared head selected . . . 18
6.11 Positive controls, and why the set is small . . . 18
6.12 Coding-consequence enrichment: a check that needs no truth set . . . 19
6.13 What this does and does not establish . . . 20
7. Open items . . . 20
Appendix: figures . . . 21

UK Biobank cholesterol application

Genome-wide fine-mapping of UK Biobank total cholesterol over LDetect EUR blocks, comparing four methods on identical loci:

method	annotations	how it uses them
`fb_xregion`	Borzoi-102	LassoNet — sparse **selection**
`funmap`	Borzoi-102	random effects — **shrinkage**, no selection
`polyfun_sniff`	Borzoi-102, quintile-binned (409)	S-LDSC prior on the same features
`polyfun_combined`	+ baselineLF v2.2 (596)	S-LDSC prior, standard specification

The two PolyFun variants separate the method from the annotation set: `polyfun_sniff` sees exactly what the other two see, `polyfun_combined` is PolyFun at its strongest.

Everything below is reproducible from public data. No UK Biobank application or credentials are needed: all three inputs are openly hosted.

---

1. Data

1.1 GWAS summary statistics

https://storage.googleapis.com/seqnn-share/sniff/_20260810_tmp_files/fixed_n_337k/sample_munged_gwas/biochemistry_Cholesterol.sumstats_munged.parquet (526 MB)

UK Biobank total cholesterol, 15,221,393 variants on all 22 autosomes, max |Z| = 82.9, 31,962 variants at p < 5e-8, median MAF 0.033.

Provenance. The `biochemistry_Cholesterol` naming, and the sibling phenotypes in the same bucket (`body_HEIGHTz`, `bmd_HEEL_TSCOREz`, `bp_DIASTOLICadjMEDz`, `pigment_HAIR`, `repro_MENARCHE_AGE`, …), identify this as the **Alkes Price group's 15-trait UK Biobank set** used in PolyFun (Weissbrod et al. 2020) — the Loh et al. BOLT-LMM summary statistics computed on UK Biobank **white-British unrelated** individuals.

Ancestry consistency (checked). The GWAS cohort (UKB white British), the LD panel (§1.2, UKB white British unrelated) and the LDetect blocks (§1.3, 1000G EUR) are all European. GWAS and LD panel are the same cohort, so the LD is in-sample rather than merely ancestry-matched.

Sample size. The `N` column reads **373,220** and that is what PolyFun itself used on this data (§1.4). We use `n = 373220`. Note the LD panel holds ~337,000 individuals, so the panel is a subset of the GWAS sample; the directory name `fixed_n_337k` refers to the panel, not the GWAS. The earlier pilot used 337,000 — the difference is 10.7% in `n` and has only a small effect on the Bayes factors, but 373,220 is the GWAS size and the value the comparator used.

1.2 LD

```
s3://broad-alkesgroup-ukbb-ld/UKBB_LD/{window}.npz   (~0.94 GB each)
s3://broad-alkesgroup-ukbb-ld/UKBB_LD/{window}.gz    (variant table)
aws s3 cp --no-sign-request ...      # public, no credentials, no requester-pays
```

UK Biobank in-sample LD from the Alkes Price group (https://registry.opendata.aws/ukbb-ld/), computed on ~337,000 unrelated white-British individuals. Windows are `chr{c}_{k*1e6+1}_{(k+3)*1e6+1}` — 3 Mb wide, stepping 1 Mb, so consecutive windows overlap by 2 Mb. `.npz` is a COO float32 **lower triangle**; the loader symmetrises it and sets the diagonal to 1.

This is the same LD resource Funmap's own paper uses for its cholesterol example, so all three methods share one LD definition.

262 windows, ~246 GB.

1.3 Loci

LDetect EUR approximately-independent LD blocks (Berisa & Pickrell 2016), shipped with this package at `inst/extdata/gwfm_regions_ldetect_EUR.csv` (1,701 blocks).

1.4 Annotations

https://storage.googleapis.com/seqnn-share/sniff/processed_borzoi_parquets/borzoi.%7BCHR%7D.annot.parquet (0.2–1.3 GB each, ~15 GB total)

Borzoi-102: 102 principal components of Borzoi variant-effect predictions over 6,385 datasets — ATAC 20, DNASE 20, RNA 20, CHIP 20, RNAGTEX 13, CAGE 9. Public GCS, no authentication. Used by all four methods.

```
.../sniff/processed_baselineLF_parquets/baselineLF2.2.UKB.{CHR}.annot.parquet   (~5 GB total)
```

baselineLF v2.2: 187 annotations, used only by `polyfun_combined`. It shares the Borzoi files' variant set and row order exactly — verified on chr22: identical SNP, BP and A1 sequences, 271,699 rows each — so `00b2` combines them by column concatenation into 289 annotations, asserting the alignment rather than assuming it. baselineLF has no tissue-specific annotations.

1.5 PolyFun prior — computed here, not downloaded

The prior is computed from our own annotation set by `00d-00g`, using the authors' `polyfun.py`. It is **not** downloaded.

Precomputed priors do exist in the same bucket

```
sniff_results_ukb_15/{sniff,baseline}/biochemistry_Cholesterol.{CHR}.txt.gz
```

and they are for the **same GWAS** — verified on chr22: N, Z and MAF agree to within text rounding across 209,974 clean 1:1 matches on position+alleles, with zero allele flips. But **which annotation set produced them cannot be established**: the bucket holds only stage-2 intermediates (30 `snpvar_bin` columns), not the annotation LD-scores, and a scalar `SNPVAR` cannot be reverse-engineered. `00c` can still fetch them as a cross-check against our own, which is the only honest use for them.

Reference data for computing it ourselves (0.30 GB, from the Zenodo record "S-LDSC reference files", 10515792 — the old `alkesgroup.broadinstitute.org/LDSCORE` URLs 404 after redirect):

```
1000G_Phase3_plinkfiles.tgz            0.29 GB  -> 1000G_EUR_Phase3_plink/
1000G_Phase3_weights_hm3_no_MHC.tgz    0.01 GB
```

Two deliberate choices.

LD-scores from the 1000G EUR panel, not the UKB LD. Computing them from the UKB LD would mean all 2,763 regions, **3.06 TB** of transfer (mean 1.11 GB each). LD-scores are a genome-wide summary rather than a per-locus quantity, and a reference panel is the conventional S-LDSC choice. The UKB LD is still used for the fine-mapping itself, where it matters. EUR is the right panel: the GWAS is UK Biobank white-British and the loci are LDetect EUR blocks.

The PolyFun annotations are binned, and why

PolyFun does **not** regress on the raw Borzoi PCs. Two properties of the raw files made that untenable, both measured rather than assumed (chr22):

- every one of the 102 PCs is signed — values span `[-841.8, 4471]` — so the per-column sums run from `-5.6e5` to `1.16e6`; and
- neither the Borzoi set nor baselineLF contains a `base` column, nor any constant column at all (5 of baselineLF's 187 also carry negatives).

Those column sums are `M`, which S-LDSC reads as a per-annotation SNP count: `M_tot = sum(M)` sets the regression weights in `LD_Score_Regression._update_func`, and the per-annotation `M_j` feed the h2 and enrichment outputs. A negative count is meaningless, and with no `base` column nothing absorbs the genome-wide mean per-SNP heritability. The failure would not have been loud: `SNPVAR = annot · taus` would still have been produced, from a regression weighted by a quantity that was not a count.

So each PC is replaced by **quintile indicators** and an all-ones `base` is added — the standard S-LDSC treatment of a continuous annotation, and exactly what the published baselineLD/baselineLF files contain (`base` there is literally all ones, so constructing one is a definition, not invented data). The lowest quintile of each PC is dropped as the reference category, because all five indicators of a PC sum to the all-ones vector and keeping all five alongside `base` is exactly collinear.

set	annotations
`sniff`	`base` + 102×4 quintile indicators = **409**
`combined`	the same + baselineLF v2.2 as published = **596**

Breakpoints are **genome-wide**, not per chromosome: S-LDSC is a single genome-wide regression, so a per-chromosome quintile would make "top quintile of ATAC-PC0" mean a different thing on each chromosome. `00h1` pools each PC across all 22 chromosomes one column at a time (~70 MB resident rather than the ~7 GB the full matrix needs); `00h2` applies them. baselineLF's own columns are kept exactly as published, including the 5 with negative values — those are its standardised continuous annotations, and re-binning them would depart from the published model.

The analysed block set: 254 of 265

Borzoi annotates no variants at all in three regions, and blocks overlapping them cannot be fine-mapped on equal terms:

- the start of the extended MHC (chr6 ~25.7 Mb onward)
- the chr8p23.1 polymorphic inversion / defensin cluster (~7.2–10.5 Mb)
- the chr11 centromere (~47–58 Mb)

The rule. A block is analysed only if it contains **no contiguous stretch longer than 100 kb in which GWAS variants exist but none are annotated** (`PREP_MAX_ANNOT_GAP_BP`). That excludes 11 of 265, leaving **254**.

The criterion is a gap rather than a retained fraction on purpose. The unannotated variants are present in both the GWAS and the LD panel; prep drops them only for want of annotations. So the methods receive a locus with a contiguous chunk missing, in which the retained variants flanking the hole appear mutually LD-linked without the intervening ones. If the causal variant lies in the hole, its proxies inside the retained set carry the signal and the credible set excludes the truth confidently — the same class of failure as handing an RSS model `r = 1` pairs with discordant `z`. A locus thinned uniformly to 60% is sound; a locus missing a contiguous 1 Mb is a truncated search space, and the retained fraction cannot tell the two apart. An earlier retained-fraction rule (≥25%) kept three blocks whose holes were 1.00, 0.72 and 0.18 Mb because their overall coverage read 58–72%.

The threshold is not doing fine-grained work: of the 265 blocks, 254 have a largest gap of ~0 (a handful of variants at most) and 11 have holes from 0.18 Mb up to the entire span. Nothing falls between 0.00 and 0.18 Mb.

excluded	span	largest hole	retained
ldEUR_11_0029	chr11:47.0–49.87 Mb	whole span	0 / 13,842
ldEUR_11_0030	chr11:50.0–53.0 Mb	whole span	0 / 9,182
ldEUR_11_0031	chr11:54.70–55.08 Mb	whole span	0 / 2,338
ldEUR_08_0012	chr8:9.15–9.64 Mb	whole span	0 / 3,903
ldEUR_08_0013	chr8:9.64–10.46 Mb	whole span	0 / 6,617
ldEUR_06_0020	chr6:25.68–26.79 Mb	whole span	0 / 6,632
ldEUR_08_0011	chr8:7.15–9.15 Mb	~2 Mb	176 / 9,011
ldEUR_11_0032	chr11:55.08–58.0 Mb	1.75 Mb	4,075 / 17,120
ldEUR_11_0028	chr11:44.7–47.0 Mb	1.00 Mb	61.8%
ldEUR_08_0015	chr8:12.0–14.2 Mb	0.72 Mb	57.8%
ldEUR_06_0019	chr6:24.9–25.7 Mb	0.18 Mb	71.8%

`ldEUR_08_0012` carries the GWAS lead with |Z| = 18.05, so this is not a negligible corner of the data and the exclusion should be stated explicitly rather than implied by a block count.

How it went unnoticed at first: prep marked all six zero-variant blocks `DONE`, because every assertion in `prep_block.py` passes vacuously at zero rows and pandas' `Series.max()` returns `NaN` rather than raising on an empty frame — so `meta.json` recorded `max_abs_z = NaN` and `preflight` reported "all 265 blocks prepared". Both now enforce the rule and report exclusions separately from failures.

Half the per-SNP heritabilities come out negative

A real property of this annotation set, worth reporting — but **not** what broke the pipeline. An earlier version of this section claimed it was; that was wrong and is corrected below.

With `base` plus 4 quintile indicators per PC, **every SNP belongs to ~102 annotation categories at once** (one per PC, bar those in a reference quintile), so its per-SNP heritability is

```
SNPVAR = tau_base + sum over 102 PCs of tau(PC, that SNP's quintile)
```

— 102 signed coefficients summed per SNP. Measured on chr1 of the stage-1 ridge fit:

	per-SNP h^2	negative	annotation taus
sniff (409)	median 1.10e-08, range [-1.79e-07, 3.72e-07]	**498,899 / 1,198,510 = 41.6%**	211 of 409 negative, range ±2.4e-08
combined (596)	median **-3.73e-10**	**605,712 / 1,198,510 = 50.5%**	302 of 596 negative

The taus are small and near-symmetric about zero, so their sum is comparable to tau_base and about half the per-SNP variances come out negative — and a variance cannot be negative. A standard S-LDSC set does not behave this way, because a SNP occupies few of its categories and base dominates the sum.

What this does and does not mean. PolyFun anticipates some negatives: that is what min_snpvar = SNPVAR.max() / q (--q, default 100) and the _constrained outputs exist for. Half is well beyond that design intent, so the prior ultimately rests on heavily clamped values, and any PolyFun number from this run carries that caveat. But the fit itself is sound and the procedure does complete — see below — so this is a caveat on interpretation, not a failure.

Correction. Stage 3 (--compute-h2-bins) failed repeatedly at ldsc_polyfun/regressions.py:348 with FloatingPointError: invalid value encountered in divide on cat / tot, and that was initially attributed to these negative heritabilities collapsing the non-negative least squares. That attribution was wrong, and two runs were spent on it (including pinning the bin count, which changed nothing). diag_polyfun_stage3.py loaded the identical inputs and found:

	sniff	combined
M zeros / non-finite	0 / 0	0 / 0
design matrix NaN / inf / all-zero columns	0 / 0 / 0	0 / 0 / 0
cond(X'X)	1,050	3,847
unconstrained OLS tot	0.410	−0.027
scipy NNLS tot	**0.433**	**0.378**

Finite, well-conditioned, and plausible heritabilities for cholesterol. The cause was the solver: PolyFun's default non-negative fit is not NNLS but an approximation to it, Lasso(alpha=1e-100, positive=True, precompute=xtx) in LstsqJackknifeSlow.__init__. Stage 3 now passes --nnls-exact, which substitutes scipy.optimize.nnls in the delete-value loop as well as the full-sample estimate. (The Lasso path was never proven to be the source of the NaN — it could not be reproduced on sklearn 1.5.1, and this environment runs 1.9.1 — so the exact solver is used on its own merits: documented as more accurate, and demonstrably finite here.)

Outcome. With --nnls-exact, stage 3 completes in ~11 minutes and both priors are produced for all 22 chromosomes:

set	SNPVAR range (chr22, 209,990 SNPs)	max/min
sniff	[1.178e-08, 9.444e-07]	80.2
combined	[5.456e-09, 3.626e-07]	66.5

Both strictly positive and finite, and both inside the ratio --q (default 100) permits — so the truncation is not saturated and the spread is not purely an artefact of the clamp.

So the methods do differ in what they ingest — fb_xregion and funmap take 102 dense continuous annotations directly, while PolyFun needs them discretised into categories and yields a per-SNP heritability that is an unconstrained sum over however many a SNP occupies — but that is a difference in representation and interpretation, not a claim that PolyFun cannot be run on them.

This weakens the polyfun_sniff framing, and the weakening belongs in the paper. polyfun_sniff no longer sees exactly what fb_xregion and funmap see; it sees a binned representation of the same 102 underlying features. The comparison still holds the feature set fixed, but no longer the representation, so it isolates the method less cleanly than first intended. The alternative — handing S-LDSC a design matrix whose implied SNP counts are negative — is not a comparison at all.

Two annotation sets, run separately. `ANNOT_SET=sniff` gives PolyFun the Borzoi features only — non-standard for S-LDSC, since enrichment estimates may be biased without baseline controls, but it means PolyFun sees the same features as `fb_xregion` and `funmap`. `ANNOT_SET=combined` adds baselineLF v2.2, the standard specification and PolyFun at its strongest. Running both removes the objection either way: if `polyfun_combined` wins, the extra-annotation advantage is visible rather than hidden; if `polyfun_sniff` already wins, the extra annotations were not what mattered.

Each set gets its own LD-scores (`ldscores_<set>/`), prior (`pf_prior_<set>/`) and fine-mapping output (`polyfun_out_<set>/`).

S-LDSC is a genome-wide regression, so this runs over all 22 chromosomes. Restricting it to the 265 significant blocks would be selection on the outcome.

1.6 Software (all upstream, unmodified)

Functional BEATRICE	this project, `main` at **`7a46a13`** or later. Anything older lacks the `reformat_memo` max-offset fix and silently produces diluted PIPs, so `03_fb_xregion.pbs` tests the checkout for the fix by content and refuses to run without it.
Funmap	https://github.com/LeeHITsz/Funmap
PolyFun	https://github.com/omerwe/polyfun
BEATRICE (if run)	https://github.com/sayangsep/Beatrice-Finemapping

Do not use the copies under `Beatrice_annot/alt_methods/` — the Funmap copy there has its input readers rewritten (originals commented out). The algorithm is untouched, but comparators must be the authors' code.

---

2. Preprocessing, exactly as performed

Step A — score every LDetect block (`00a_score_blocks.py`)

For each of the 1,701 LDetect EUR blocks, take `max(Z^2)` over the summary statistics inside it and convert to a two-sided p. Writes `blocks_scored.csv`.

Step B — select loci and assign LD windows (`00_make_blocks.py`)

Keep blocks with `min_p < 1e-6`: **265 blocks**.

For each, choose the 3 Mb UKBB window that (a) contains the block's **lead variant** and (b) retains the most of the block, then trim the block to that window.

> A block up to 2 Mb always fits some window; wider ones may not, because window boundaries sit on integer Mb. **37 of the 265 need trimming: 18.81 Mb total, 3.89% of the span, with zero lead variants lost.**
>
> An earlier version of this pipeline silently dropped all 58 blocks wider than 3 Mb. **13 of them pass p < 1e-6**, including `ldEUR_11_0034` (chr11:58.8–62.2 Mb, lead |Z| = 19.1 — the **FADS1/FADS2** cluster) at p = 2.4e-81 and `ldEUR_17_0039` (lead |Z| = 14.3, the **ABCA6/8/9/10** cluster) at p = 1.9e-46. Dropping them would have removed two of the strongest cholesterol loci in the genome. They are now trimmed like any other wide block and retained.
>
> The alternative — stitching two overlapping windows and zeroing pairs more than 2 Mb apart — was rejected: it keeps more variants but makes the LD inexact for exactly the long-range pairs a 3 Mb locus exists to model.

Writes `blocks.csv` (with `orig_start`, `orig_end`, `trimmed_bp`, `lead_bp`, `lead_abs_z` per block) and `ld_windows.txt`.

Step C — build per-block inputs (`prep_block.py`)

Per block:

1. Read the LD panel's variant table; restrict to the block and to **biallelic SNVs**; drop duplicate positions entirely.
2. Match to the summary statistics on **position + unordered allele pair**, and flip `Z` wherever the sumstats' A1 differs from the LD panel's A1, so `Z` is harmonised to the LD allele coding.
3. Join the Borzoi annotations on the **ordered** allele pair, as coded by the LD panel.
4. Take the LD submatrix **by the row index carried through every merge**, symmetrise, set the diagonal to 1, store float32.
5. Standardise annotations per block (mean 0, sd 1) and clip to ±10.

Each of those rules exists because of a bug that actually occurred:

- **Row index, never position.** Position is not unique. A position-keyed LD lookup hands every allele at a site the same LD row, producing r = 1 pairs carrying different z — logically impossible input, and the worst case for an RSS model. It passed every global diagnostic (`estimate_s_rss` s = 0.01–0.03) while capturing up to 100% of the posterior mass.
- **Ordered allele pair for annotations.** Borzoi lists some indels in both allele orders with different values; an unordered key silently duplicates rows.
- **Biallelic SNVs, no duplicate positions.** What Funmap's own pipeline does. Removes the problem at source, at a cost of ~10% of variants.

`prep_block.py` asserts no two retained variants share an LD row. Verified on the pilot blocks: zero duplicate positions, and max |dZ| = 0.000 within every genuine r = 1 group.

Resulting scale

265 blocks, 464 Mb, ~2.36M variants, median p 8,997, max p 15,219.

---

3. Running

Bootstrap once

```
# Put APP_ROOT on project space, not $HOME. Peak usage is ~429 GB:
#   246  UKB LD .npz/.gz, needed right through stage 05
#    94  prep locus_ld.npy (float32 submatrix per block, as well as the windows)
#    40  LD-scores for the two annotation sets
#    40  annotations (Borzoi + baselineLF + combined)
#     9  everything else
export APP_ROOT=/rds/general/project/<project>/live/$USER/chol_app

# Modules the COMPUTE nodes need - without these the venvs will not start
# there, since they are built against whichever python is active now.
# NB `module avail` indents, so an anchored pattern like ^R/ never matches.
module avail 2>&1 | grep -E "^ *(Python|R|awscli)/"
# On Imperial CX3 these two share GCCcore-12.3.0 (gfbf-2023a IS GCC 12.3.0),
# so they co-load. Python 3.11 is what the PolyFun pins below were validated
# against; R 4.3.2 is close to susieR 0.11.92's era, which is the one package
# here that must be an old version.
# tools/prod FIRST: on CX3 these modules are hierarchical and are not
```

```
# loadable until it is in place ("exist but cannot be loaded as requested").
export APP_MODULES="tools/prod Python/3.11.3-GCCcore-12.3.0 R/4.3.2-gfbf-2023a"

# awscli goes in its own variable, not APP_MODULES: it is typically built
# against a different toolchain (GCCcore-13.2.0 against Python's 12.3.0) and
# the two may refuse to load together. Only stage 01 needs it, so it is
# loaded there alone.
export AWS_MODULE="awscli/2.17.54-GCCcore-13.2.0"   # tools/prod comes from APP_MODULES

# Sanity-check before going further - if these do not both work, the jobs
# will fail in a way that is tedious to diagnose later:
module purge && module load $APP_MODULES && python3 -V && R --version | head -1
module purge && module load $AWS_MODULE && aws --version
module purge && module load $APP_MODULES

bash setup.sh
```

`setup.sh` creates the directory tree, downloads the summary statistics, clones upstream Funmap and PolyFun, builds the three Python environments, installs susieR 0.11.92 for PolyFun, and writes `$APP_ROOT/env.sh`. It is safe to re-run: every step skips work already done.

Three environments are needed because the pins genuinely conflict:

env	for	pins
`APP_VENV`	prep, fb_xregion	numpy, pandas, pyarrow, scipy, torch
`FUNMAP_VENV`	Funmap	numpy 1.26.2, scipy 1.11.4, pandas 2.1.3, matplotlib 3.8.2
`POLYFUN_ENV`	PolyFun	numpy<2, pyarrow 17, rpy2 3.5.17, plus susieR 0.11.92 in `POLYFUN_RLIB`

Then put functional-beatrice at `$FB_DIR` — **`main` at `d0fb022` or later**; `03_fb_xregion.pbs` verifies this by content and refuses to run otherwise.

Where to run things

`setup.sh` is run **from the cloned repo** — it copies every script into `$APP_ROOT`. Everything after that is run **from `$APP_ROOT`**:

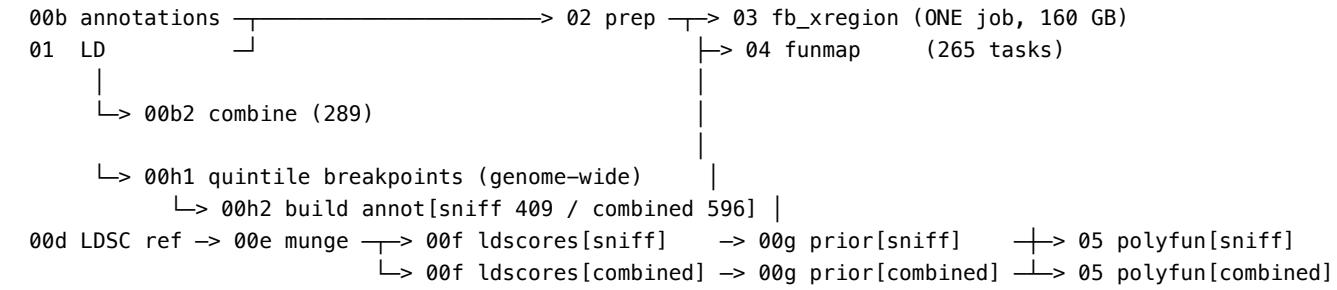
```
cd "$APP_ROOT"
```

Strictly, the shell scripts all `cd "$APP_ROOT"` as their first action and the python scripts resolve `APP_ROOT` from the environment, so they work from any directory as long as the path you give resolves. But `$APP_ROOT` is where the copies live, so running there is what the commands below assume.

Submit

```
cd "$APP_ROOT"
source "$APP_ROOT/env.sh"
bash submit_all.sh --dry-run     # inspect the qsub commands and dependencies
bash submit_all.sh
```

`submit_all.sh` chains the stages with PBS dependencies, so nothing starts before its inputs exist:



`00d-00g` compute each PolyFun prior from our own annotations and run in parallel with the LD download. `00f`, `00g` and `05` are each submitted twice, once per `ANNOT_SET`; `submit_all.sh` does this for you.

The three downloads run concurrently. 03, 04 and 05 are independent of one another. `--from 03` resumes at a stage with dependencies dropped.

Check before committing CPU

```
python preflight.py data      # before prep:    downloads complete and non-truncated?
python preflight.py prep      # before methods: all 265 blocks built?
python preflight.py methods   # progress per method
```

`preflight.py` exits non-zero and names what is missing. The point is to fail in seconds rather than a third of the way through a 265-task array.

Collect

```
python 06_collect.py && python preflight.py collect
python 07_plot_blocks.py                        # the 4 strongest blocks
python 07_plot_blocks.py ldEUR_01_0067          # or named blocks
python 07_plot_blocks.py --n 8                  # or the 8 strongest
```

`07_plot_blocks.py` skips blocks that are not finished rather than failing, so it is usable while a run is still going.

`06_collect.py` is deliberately **not** chained, so a partial failure in one method does not quietly produce a half-empty results table.

Every stage is idempotent — a block with a `DONE` marker or an existing output is skipped — so any array can be resubmitted after a partial failure. If you change the p-value threshold, re-run `00a/00` and update the `#PBS -J` array sizes to match the new block and window counts.

---

4. Diagnostics

Every stage sources `lib.sh`, which gives it:

- a **banner** recording job id, host, cpus, cwd, free disk, every relevant environment variable, and — after the venv is activated — the python that actually ran and its package versions. Most failures are an environment pointing somewhere unexpected, and this makes that visible immediately.
- an **ERR trap** naming the failing line number, the command, the working directory and the elapsed time. No more silent `set -e` exits.

- **preconditions** (`require_file`, `require_dir`, `require_env`) checked before expensive work. `require_file` takes a minimum size, because the failure that actually happens is a truncated download that is present but unusable.
- **output validation** (`check_output`) after it, so a stage cannot report success having written a zero-row file. `01` opens each `.npz` to prove it loads; `00f` checks the LD-score file has rows and annotation columns; `00g` asserts all 22 chromosomes exist and that SNPVAR is finite and positive; `04` checks funmap wrote one PIP per input variant.

Abnormal lines are prefixed `FATAL` or `WARN`, so an entire array triages with:

```
bash triage.sh            # all stages
bash triage.sh 02_prep    # one stage
```

It separates completed tasks, tasks that failed with a `FATAL` (showing the message), and tasks with neither — which means the scheduler killed them, so look at walltime and memory. It also groups the distinct failure modes, which is what tells you whether 40 failures are one problem or forty.

A worked example of what a failure looks like:

```
[14:02:11] chol_prep[57]  ===== start =====
[14:02:11] chol_prep[57]  host     : cx3-6-21.cx3.hpc.ic.ac.uk  (4 cpus)
[14:02:11] chol_prep[57]  APP_ROOT : /rds/general/user/xx/home/chol_app
[14:02:11] chol_prep[57]  disk free: 812 GB
[14:02:12] chol_prep[57]  python   : .../appenv/bin/python (Python 3.11.6)
[14:02:12] chol_prep[57]  block    : ldEUR_04_0048  chr4  window chr4_68000001_71000001
[14:02:12] chol_prep[57]  FATAL missing file: ld/chr4_68000001_71000001.npz
[14:02:12] chol_prep[57]        (stage 01 incomplete for this window)
```

---

5. Resources

Measured (5 pilot blocks, 35,621 variants, 2000 joint iterations): 366 s wall with `.npy` LD against 493 s with text; peak RSS 3.25 GB of which 1.06 GB is the retained float32 LD.

Projected for 265 blocks: LD 94 GB + annotations ~1 GB + ~2 GB fixed overhead. **Request 160 GB for `03_fb_xregion`**; mini-batching is not needed.

Measured per-method, on one block (`ldEUR_20_0038`, chr20, p = 4,238, 8-core laptop):

method	block	p	wall time
fb_xregion	5 blocks **jointly**, 2000 iterations	35,621 total	366 s
Funmap	ldEUR_22_0024_part	7,777	**702 s (11.7 min)**
Funmap	ldEUR_20_0038	4,238	3,107 s — see below
PolyFun `finemapper.py`	ldEUR_20_0038	4,238	46 s (6–8 s in SuSiE)
PolyFun `finemapper.py`	ldEUR_22_0024_part	7,777	29 s

The 3,107 s figure is not a clean measurement — that run spent most of its life competing with a stray process for the CPU. The clean number is chr22: **11.7 minutes at p = 7,777**, despite that block being larger and taking more stage-2 iterations (19 vs 13). Funmap's cost tracks iteration count to convergence as much as p, so it varies block to block.

Budget on the clean figure: roughly **15–25 min per block** at the median p ≈ 9,000, more for the largest. A 12 h walltime on `04_funmap.pbs` is ample, and it parallelises trivially as an array job.

Still to measure: fb_xregion total runtime. `regularize_ld` always falls through to a full eigendecomposition on real float32 UKBB LD (min eigenvalue ≈ −0.029, so the Cholesky fast path never succeeds), which is $O(p^3)$ per block. At p ≈ 9,000–15,000 over 265 blocks this dominates setup. Run `03` against a 10-block manifest and extrapolate before trusting the walltime.

Verified end to end

All three methods were run on `ldEUR_20_0038` with identical inputs:

ldEUR_20_0038 (chr20:62.19–62.97 Mb, p = 4,238, max |Z| = 6.05)

method	sum PIP	max PIP	top variant	|Z|	rank
fb_xregion	1.55	0.561	62,362,116	6.05	1
funmap	9.99	**1.000**	62,370,732	6.00	13
polyfun_susie	9.04	0.407	62,370,732	6.00	13

This block holds **two independent signals** of near-identical strength 534 kb apart — peak A at 62.36 Mb (max |Z| 6.05) and peak B at 62.91 Mb (max |Z| 6.04), r = 0.097 between their leads. All three methods now settle on peak A. In blocks with multiple peaks, report per-signal rank rather than global rank: a correct call on a second signal scores badly on a global |Z| rank.

The LD regularisation defect (fixed)

Before `d0fb022`, `regularize_ld` added `(1e-3 - min(eig))` to the LD **diagonal**, scaling every implied correlation by `1/(1+c)`. On ldEUR_01_0067 (min eig −0.069) that turned r = 0.996 into 0.931 — and the approximate Bayes factor has a crossover near r = 0.955, above which co-inclusion of collinear variants is penalised (−11.1 nats at 0.996) and below which it is **favoured** (+8.2 nats at 0.931). 100% of sampled pairs with raw |r| > 0.955 were pushed across it.

The effect was real and visible: at SORT1, the single highest-weighted configuration in the memo was `{rs12740374, rs7528419}` (r = 0.996), 71% of posterior mass sat on configurations containing both, and PIP summed to 1.97 across the four variants at r > 0.99.

Clipping the eigenvalues and renormalising the diagonal fixes it:

	sum over r>0.99	rs12740374	rs7528419	rs646776	rs629301
diagonal shift	1.97	0.856	0.772	0.184	0.160
eigenvalue clip	**1.00**	**0.942**	0.059	0.002	0.000
PolyFun (published)	1.02	0.917	0.064	0.022	0.016

rs12740374 is the functionally validated causal variant for this locus (Musunuru et al., Nature 2010). The four variants are in r = 0.996–1.000 LD with |Z| spanning only 35.04–35.14, so the marginal statistics cannot separate them — and fb_xregion and PolyFun, independently, both put ~0.92–0.94 on the validated one.

The benchmark is unaffected. The shift only fired when the LD was not positive definite. Simulated LD is PD (min eig +0.042) and takes the Cholesky fast path; `examples/data` reproduces bit-identically after the change.

ldEUR_22_0024_part (chr22:50.00–51.24 Mb, p = 7,777, max |Z| = 5.52)

method	sum PIP	max PIP	top variant	|Z|	rank
fb_xregion	1.00	0.033	50,858,813	5.46	2
funmap	9.99	**1.000**	50,858,813	5.46	2
polyfun_susie	9.97	0.133	50,838,241	5.42	11

Spearman between PIP vectors on chr20: funmap–polyfun 0.722, fb_xregion–polyfun 0.533, fb_xregion–funmap 0.518.

Three things to look at before reading anything into the full run:

1. **Funmap returns PIP of exactly 1.000**, on the lead variant in both blocks and on two variants in chr20. At |Z| ≈ 6 inside a region of several thousand variants in LD, a posterior of exactly 1 is not credible; it suggests numerical saturation rather than certainty.
2. **fb_xregion is far more conservative** — sum(PIP) 1.0–1.7 against ~10 for the other two. Across five prepared blocks its confidence tracks signal strength: max PIP 0.86 at |Z| 35, 0.94 at |Z| 10, 0.73 at |Z| 6, but only 0.16 at |Z| 5.0 and 0.03 at |Z| 5.5. Being unconfident at |Z| 5.5 among 7,777 variants in LD is defensible; funmap returning exactly 1.000 on the same variant is the harder number to justify. Which of the two is better calibrated is an open question, and not one to settle by assuming the confident method is right.
3. **Funmap and PolyFun both put sum(PIP) ≈ 9.99**, their L = 10 prior, in every block. That is SuSiE-like behaviour and means the posterior expected number of causal variants is essentially the prior.

A real 50 kb gap at chr22:50,364,571–50,414,983 is visible in the z-scores. It is genuine: zero variants there in both the summary statistics and the LD panel, not an artefact of the filtering.

Building the PolyFun environment

`polyfun.yml` (python 3.8, `r-susier==0.11.92`) solves on x86_64 Linux, which is what the cluster runs. It does **not** solve on osx-arm64. A working pip environment needs four specific pins, each found the hard way:

pin	why
`pyarrow<=17`	newer pyarrow requires numpy ≥ 2, which conflicts with the rest
`rpy2==3.5.x`	3.6 turned `numpy2ri.activate()` into a raised `DeprecationWarning`, and finemapper.py calls it
`susieR 0.11.92`	0.12+ moved `bhat/shat/R/var_y` out of `susie_suff_stat` into `susie_rss`; finemapper.py calls the old signature. Install with `remotes::install_github("stephenslab/susieR@v0.11.92")` — note the `v` prefix on the tag
R dependency tree	susieR's deps must come from a **CRAN snapshot**, not today's CRAN. Current `Matrix` (1.7-6) declares `R (>= 4.4)`, so under `R/4.3.2` it is filtered out of `available.packages()` and reported as `dependency 'Matrix' is not available` — taking `irlba`, then `mixsqp`, then `susieR` down with it. `setup.sh` resolves from `packagemanager.posit.co/cran/2023-11-01` (override with `R_CRAN_SNAPSHOT`), giving `Matrix 1.6-1.1`, `irlba 2.3.5.1`, `mixsqp 0.3-48`, `ggplot2 3.4.4`. This also avoids compiling a 2026 tree (ggplot2 4.x, S7, RcppArmadillo 15.x) for a 2022 package
`mixsqp`	The one dependency taken from **current** CRAN, not the snapshot. Snapshot `mixsqp 0.3-48` declares `CXX_STD = CXX11` in `src/Makevars`, so it builds with `-std=gnu++11`, and the Armadillo 12 headers bundled by RcppArmadillo 0.12.x hard-error: `*** C++14 compiler required.` Upstream `0.3-54` drops the `CXX_STD` pin for exactly this reason and is otherwise identical (same Depends/Imports/LinkingTo); susieR 0.11.92 imports `mixsqp` unversioned. `setup.sh` also sets `R_MAKEVARS_USER` with `CXX11STD = -std=gnu++14` as a safety net for any other CXX11-pinned package of that era
`RPY2_CFFI_MODE=ABI`	only if the rpy2 wheel was built against a different R minor version than the one installed

Set `R_LIBS` so the 0.11.92 library comes first, and `R_HOME` to the R installation rpy2 should embed.

---

6. Results

Produced by `06_collect.py` (collection and QC), `09_ld_conditioning.py` (per-block LD diagnostics), `10_analyse_pips.py` (the tables below), `diag_comparator_ladder.py` (the ablation audit) and `11_export_results.py` (the local bundle).

254 blocks, 7 columns, 15,375,983 (region, variant, method) rows, **zero missing outputs**. The 11 blocks excluded by the unannotated-gap rule (§1) are absent by construction, not missing: `ldEUR_06_0019/0020`, `ldEUR_08_0011/0012/0013/0015`, `ldEUR_11_0028/0029/0030/0031/0032`.

6.1 What the seven columns are

Four are methods; three are states of our own code and must never be quoted as methods. The three functional-BEATRICE columns are **not a ladder** — they are two ablation arms off the full method, and the commit graph is the authority:

```
upstream sayangsep/Beatrice-Finemapping @ 02dd5af  ──────────────►  beatrice
  │  no annotations, no shared prior head,
  │  upstream defaults: sigma_sq 0.05, sparse_concrete 50, n_caus 20
  │
  │  our fork adds, before any of the columns below:
  │    • 102 Borzoi/Sniff annotations per variant
  │    • a LassoNet prior head SHARED across all 254 blocks (one set of
  │      annotation weights for the genome, not one per block)
  │    • tuned hyperparameters (f9b089a): n_caus 20→10, sigma_sq
  │      0.05→0.0899, sparse_concrete 50→51
  │    • a single-logit prior head (de18eda)
  │
8d59368   memo offset: mean → max  (the log-sum-exp trick)
74c76a1   LD: project onto the nearest PD correlation matrix, fixed 1e-3 floor
  │
d0fb022  ──────────────────────────────────────────►  fb_prefix
  │
fc82a9c   pass the LEARNED per-variant prior into the credible-set sweep;
  │       make the LD eigenvalue floor ADAPTIVE (the panel's own error scale)
f3bd29a  ──────────────────────────────────────────►  fb_xregion
  │
9494664   ABLATION: revert ONLY the memo offset, max → mean
        ──────────────────────────────────────────►  fb_memomean
```

`f3bd29a` is an **ancestor** of `9494664`, so `fb_memomean` sits downstream of `fb_xregion`, not between it and `fb_prefix`.

column	kind	what it is, exactly
`fb_xregion`	method	functional BEATRICE at `f3bd29a`. Annotations + shared LassoNet head + max memo offset + nearest-PD LD with an adaptive floor + the learned per-variant prior carried into the credible-set sweep. **The full method.**
`funmap`	method	upstream Funmap, unmodified. `L = 10`.
`polyfun_sniff`	method	PolyFun + SuSiE, prior from the Borzoi/Sniff annotations only — the same information `fb_xregion`'s head sees, binned. `--max-num-causal 10`.
`polyfun_combined`	method	PolyFun + SuSiE, prior from Borzoi + baselineLF (baselineLF adds explicit coding and conserved categories; see §6.12).
`fb_prefix`	**state, not a method**	FB at `d0fb022`: everything `fb_xregion` has **except** the learned prior (the sweep runs under a **uniform** prior) and **except** the adaptive floor (a fixed `1e-3` clip). Two variables from `fb_xregion`.
`fb_memomean`	**state, not a method**	FB at `9494664`: `fb_xregion` with **one line reverted** — `reformat_memo` subtracts the **mean** log-BF instead of the **max**. Everything else, learned prior and adaptive floor included, is identical. **One variable from `fb_xregion`.**
`beatrice`	method (literature baseline)	upstream `sayangsep/Beatrice-Finemapping` @ `02dd5af`, unmodified, on its own published defaults. No annotations, no shared head.

What each ablation isolates, and what it does not.

- `fb_xregion` vs `fb_memomean` — **the only single-variable comparison here.** It isolates the memo normalisation and nothing else. Subtracting the mean rather than the max leaves every above-average configuration positive, so anything above `mean + log(1e15)` saturates; `calculate_pip` then returns a count of configurations rather than a posterior.
- `fb_xregion` vs `fb_prefix` — isolates **two** changes together, the learned prior pass-through and the adaptive LD floor. They were committed together in `fc82a9c` and have not been separated, so no statement here attributes an effect to one of them alone.
- `fb_xregion` vs `beatrice` — moves annotations, the shared head, three hyperparameters, the memo normalisation **and** the LD treatment at once. It is a literature baseline, not an ablation, and nothing causal should be read into the gap.

Row alignment is checked, not assumed. `06_collect.py` asserts that every method's PIP vector matches `snps.csv` in length but never in order, and upstream BEATRICE is a separate program reading a text LD file we materialise for it. `diag_comparator_ladder.py [1]` verifies the order against the labels upstream itself writes: 0 mismatches and `variant_index == 0..p-1` in all 254 blocks. Note that those labels are `"<region_id>_<BP>"`, not rsIDs (`prep_block.py:172`), so comparing them to `snps.csv` `SNP` reports 254/254 mismatched however well aligned the run is.

6.2 Headline, signal regions only

185 of 254 blocks carry a genome-wide signal (lead |Z| ≥ 5.45); 69 do not. Median lead |Z| among the 185 is 7.22.

method	med sumPIP	med maxPIP	PIP ≥ .95	med CS	top = lead	med lead PIP
`beatrice`	3.34	0.0065	1	348	2	0.0000
`fb_memomean`	32.41	0.0262	1	72	46	0.0128
`fb_prefix`	1.94	0.8797	87	2	67	0.0090
`fb_xregion`	1.01	0.9169	79	**2**	83	**0.3817**
`funmap`	9.99	1.0000	**145**	**1**	56	0.1951
`polyfun_combined`	9.98	0.4345	39	4	**88**	0.2507
`polyfun_sniff`	9.95	0.4363	46	4	76	0.2297

`funmap` makes by far the most confident calls and `polyfun_combined` picks the exact lead most often, so neither of those columns alone separates the methods. The next two tables do.

6.3 Do the confident calls carry marginal evidence?

Count variants, not regions. `10_analyse_pips.py:107` counts regions whose top variant clears 0.95, one per region. That is a useful summary but it cannot see a method that calls six variants inside one block, and here that distinction is the whole story for one column. All 254 blocks, a call being any variant at PIP ≥ 0.95:

method	variants ≥ .95	blocks	per block	med |Z|	|Z| < 2	|Z| < 5.45	in blocks with no signal
`beatrice`	1	1	1.00	82.85	0	0	0
`fb_memomean`	1	1	1.00	82.92	0	0	0
`fb_prefix`	**653**	106	**6.16**	**0.57**	**540**	563	218
`fb_xregion`	136	90	1.51	7.45	**1**	21	12
`funmap`	263	170	1.55	5.78	**61**	119	25
`polyfun_combined`	69	41	1.68	7.75	1	12	2
`polyfun_sniff`	83	49	1.69	7.64	2	17	3

`fb_prefix` declares **653 variants causal at PIP ≥ 0.95, 540 of them on |Z| < 2** — 6.16 per block, median |Z| of a call 0.57. That is exactly what saturation at `sparse_concrete` = 51 produces, and a per-region maximum is blind to it: the region view records only 106.

`funmap` makes 263 calls and 61 of them sit on variants with no marginal evidence; 119 are below genome-wide significance. `fb_xregion` makes roughly half as many calls as funmap and **one** of them is on $|Z| < 2$.

The region-level view, for completeness — each region's top variant, where that variant reaches 0.95:

method	regions	med $\|Z\|$	$\|Z\| < 2$	$\|Z\| < 5.45$
`fb_prefix`	106	6.55	19	37
`fb_xregion`	90	6.98	1	13
`funmap`	170	5.36	52	88
`polyfun_combined`	41	7.83	0	5
`polyfun_sniff`	49	8.05	1	8

Quote whichever you like, but say which one it is.

6.4 Localisation: what |Z| does each method's own top variant carry?

Signal regions. `top = lead` scores picking a perfect-LD partner of the lead as a miss, which understates localisation; the |Z| the top variant actually carries does not.

method	med top $\|Z\|$	top $\|Z\| \geq .99 \times$ lead	top $\|Z\| < 2$	top = lead
`beatrice`	0.76	6/185	153	2
`fb_memomean`	4.84	54/185	62	46
`fb_prefix`	6.25	95/185	37	67
`fb_xregion`	**6.95**	**134/185**	**1**	83
`funmap`	6.07	78/185	44	56
`polyfun_combined`	6.66	106/185	4	88
`polyfun_sniff`	6.59	99/185	3	76

Median lead |Z| is 7.22, so `fb_xregion`'s median top variant at 6.95 is essentially at the lead. It lands on a null in 1 of 185 blocks; `funmap` does so in 44.

Upstream BEATRICE's top variant carries $|Z| = 0.76$ in the median signal region and $|Z| < 2$ in 153 of 185. It does localise in the handful of extreme loci — top |Z| 82.85 against a lead of 82.92 at APOE, 35.40 against 35.40 at PCSK9 — but those are the strongest loci in the genome and nothing about them generalises.

6.5 Two different fixes, two different effects

Both arms off `fb_xregion` (§6.1) degrade it, but along different axes, and the two axes are not the same axis.

Calibration is the mean → max memo offset, on its own. The single-variable comparison is `fb_xregion` vs `fb_memomean` (§6.1) — one reverted line, everything else held fixed. Reverting it takes median max PIP from **0.917 to 0.026**, confident variants from **136 to 1**, near-lead localisation from **134/185 to 54/185**, and the median credible set from **2 to 72**. The per-region Spearman between the two is **+0.012**: with the mean offset the method does not agree even with itself. The mechanism is saturation — subtracting the mean leaves every above-average configuration positive, anything above `mean + log(1e15)` saturates, every other proposed configuration lands more than 745 nats below and is stored as exactly 0.0, and the sweep runs inert. The surviving PIP is a variational selection frequency, not a posterior.

(An earlier draft of this section compared `fb_memomean` to `fb_prefix` and called that the single-variable ablation. It is not: that pair differs by the memo offset **and** the learned prior **and** the floor, because `fb_memomean` branches off `fb_xregion` rather than sitting between the two.)

Localisation responds to the learned prior and the floor. Blocks where the top variant carries ≥ 99% of the lead's |Z|, out of 185: `fb_xregion` 134, `fb_prefix` 95, `fb_memomean` 54, `beatrice` 6. Blocks where it carries $|Z| < 2$: 1, 37, 62, 153 respectively. Read these as four separate values, **not** as a sequence — only the `fb_xregion`/`fb_memomean` pair differs by a single variable, `fb_xregion`/`fb_prefix` differs by two, and `beatrice`

differs by everything (§6.1). In particular, nothing here isolates the contribution of the annotations: no column has our annotations removed while holding the rest of the fork fixed.

The prior/floor arm (`fb_prefix` → `fb_xregion`) trades quantity for quality, and per variant (§6.3) the trade is far larger than the region-level view shows: confident calls **653 → 136**, calls on $|Z| < 2$ **540 → 1**, calls below genome-wide significance 563 → 21, calls in blocks with no signal 218 → 12, median $|Z|$ of a call 0.57 → 7.45. Median credible set goes 7 → 2 and the median PIP on the GWAS lead 0.0082 → 0.3195.

6.6 Saturation is gone, and it is measurable

`sum(PIP)` is the posterior mean number of causal variants, so a value at the model's own cap means the posterior is pinned against it rather than fitted.

method	min	median	**max**
`beatrice`	2.531	3.328	17.838
`fb_memomean`	1.590	32.511	38.475
`fb_prefix`	1.000	1.328	**51.000**
`fb_xregion`	0.640	1.002	7.510
`funmap`	1.000	9.989	10.000
`polyfun_sniff`	1.000	9.950	10.000

`fb_prefix` reaches **exactly 51.000**, our fork's `sparse_concrete` cap. `fb_xregion` tops out at 7.510, below its own `n_caus` = 10, so it never saturates in any of the 254 blocks. Funmap and PolyFun sit at 10.000, their configured `L` / `--max-num-causal`; neither adapts `sum(PIP)` to whether the block holds a signal.

6.7 Upstream BEATRICE emits almost no credible sets

Of 254 `credible_set.txt` files, **253 are zero-byte** and the largest is 4 bytes. Its `gamma_key = 0.2` gate never fires because the max PIP never reaches it. This statement depends on nothing of ours — it is the published program, unmodified, on its published defaults, at n = 373,220.

6.8 Where the methods disagree

Per-region Spearman of PIPs, median over signal regions: `fb_prefix`–`fb_xregion` +0.898, `fb_xregion`–`funmap` +0.891, `funmap`–`polyfun_*` +0.856/+0.858, `fb_xregion`–`polyfun_*` +0.810/+0.825, the two PolyFuns +0.825. `beatrice` against everything is −0.005 to +0.004 and `fb_memomean` +0.009 to +0.013.

Restricted to blocks where **both** methods are confident, funmap is the outlier:

pair	both confident	disagree on the variant
`fb_xregion` vs `polyfun_combined`	34	**8**
`fb_xregion` vs `polyfun_sniff`	38	**9**
`polyfun_combined` vs `polyfun_sniff`	38	3
`funmap` vs `polyfun_combined`	38	21
`funmap` vs `polyfun_sniff`	45	24
`fb_xregion` vs `funmap`	70	**47**

6.9 LD quality

`09_ld_conditioning.py` records each block's minimum eigenvalue and the floor the adaptive projection applied. The floor caps the attainable correlation at $\approx 1 -$ floor, so it is a distortion as well as a fix: median 0.0305, p90 0.0703, max 0.6311; 61 of 254 blocks exceed 0.05 ("distorted") and 10 have a minimum eigenvalue below −0.1 ("bad_ld"). Every table above is reproduced split by that flag in `10_analyse_pips.py`'s output. The ranking of methods does not change across the split; absolute confidence does.

Four candidate explanations for the poor conditioning were tested and **refuted**: it is not a prep bug (max |off-diagonal| = 1.00000, a single lower triangle, `R + R.T` correct), not sparsification (100% dense), not float16 storage (0.07–0.85% round-trip error, 1.95M distinct values), and not block size (Spearman −0.261 against p). Pairwise-complete estimation in the published UKBB panel remains the untested candidate. **State this as unresolved** rather than attributing it.

6.10 Which annotations the shared head selected

LassoNet selects through the skip coefficient θ (`trainer_annot.py:274`); the hierarchy constraint zeroes the nonlinear path for any feature with θ = 0, so θ = 0 means the annotation is excluded outright. **14 of 102 survive**:

rank	annotation	|θ|
1	`RNA-PC12`	2.67e-03
2	`RNAGTEX-PC3`	1.86e-03
3	`RNA-PC5`	1.44e-03
4	`ATAC-PC14`	4.39e-04
…		
10	`RNAGTEX-PC5`	4.33e-05

The leaders are all expression PCs, which is directionally sensible for a lipid trait. The **pre-registered** prediction (§7) was `RNAGTEX-PC5`, the component that separates liver at 3.5 SD. It survives selection but ranks 10th of 14, 62× below the top. With 14 of 102 surviving, membership alone is weak evidence: **record this as a miss, not a confirmation.**

6.11 Positive controls, and why the set is small

There is **no gold-standard causal variant set** for fine-mapping. The autoimmune fine-mapping catalogue states it outright, and recent genome-wide fine-mapping work deliberately evaluates with metrics that need no known causals (replication across GWAS samples). So what follows is a positive-control check, not a benchmark, and it cannot produce a precision or recall number.

`validated_variants.csv` holds seven variants with **variant-level** functional evidence — not loci, and not lead SNPs. Padding the list with well-known lead variants would make the check circular: at most lipid loci the validated variant is the GWAS lead, so "high PIP on the validated variant" would restate `top = lead` from §6.4.

Two fields decide how each result reads:

- **`tier`** — A: a coding variant whose protein-level mechanism is established (PCSK9 R46L, APOE ε2/ε4, TM6SF2 E167K, GCKR P446L). B: non-coding, with the variant itself perturbed experimentally (SORT1 rs12740374, LDLR rs6511720).
- **`identifiable`** — whether association data can single the variant out. `rs6511720` is in complete LD with three other LDLR intron-1 SNPs and the same study finds `rs57217136` equally functional, so spreading PIP across that haplotype is the **correct** answer and a low PIP there is not a miss. The SORT1 study likewise states that `rs12740374` tags a haplotype and any SNP on it could be causal.

The informative cases are the ones where the validated variant is **not** the regional lead. APOE is the clearest: the lead in `ldEUR_19_0031` is `rs1065853`, for which no variant-level functional evidence was found, while the mechanism is carried by `rs429358`/`rs7412`. `11_export_results.py` writes `validated_pips.csv` with every method's PIP and exact within-block rank for each control, alongside r^2 with the lead and the number of variants at $r^2 \geq 0.95$ and ≥ 0.99 — so a PIP can be read against how many variants are indistinguishable from it.

All seven controls fall inside the analysed blocks. PIP, with the within-block rank in brackets (`fig3_positive_controls.pdf`):

control	k at $r^2 \geq .99$	fb_xregion	funmap	polyfun_comb	polyfun_sniff	fb_prefix	beatrice
PCSK9 rs11591147	1	1.000 (1)	1.000 (1)	1.000 (1)	1.000 (1)	1.000 (1)	0.009 (1)
APOE ε4 rs429358	1	1.000 (1)	1.000 (1)	1.000 (1)	1.000 (1)	1.000 (4)	0.012 (3)
GCKR rs1260326	1	0.999 (1)	0.999 (1)	0.999 (1)	0.997 (1)	1.000 (1)	0.007 (2)
APOE ε2 rs7412	1	**0.998 (5)**	3e-4 (5147)	0.008 (70)	0.003 (289)	3e-21 (7317)	0.974 (1)
TM6SF2 rs58542926	1	0.900 (2)	0.997 (2)	0.930 (1)	0.994 (1)	**7e-13 (6740)**	**1e-13 (1384)**
SORT1 rs12740374	**4**	0.666 (2)	**1.000 (1)**	0.771 (1)	0.934 (1)	0.126 (3)	0.028 (1)
LDLR rs6511720	**8**	0.007 (9)	7e-5 (7936)	0.015 (55)	0.002 (142)	1e-7 (395)	0.011 (8)

Reading it, in order of how much each row actually discriminates:

- **The first three are sanity checks and everything passes.** Each is a uniquely identifiable variant and every method returns ≈ 1. Good — but they separate nothing, which is the circularity the short list was chosen to avoid.
- **APOE ε2 is the one genuine discrimination.** rs7412 is not the block's lead (rs1065853 is, at $r^2 = 0.9897$) and it has no near-perfect partner of its own. fb_xregion gives it 0.998; funmap 3e-4, both PolyFuns 0.003–0.008. Among the four methods only functional BEATRICE recovers a textbook causal coding variant here. **Caveat against it:** it also calls rs1065853 at ≈ 1.0, and the two are $r^2 = 0.99$ apart, so it found ε2 without choosing ε2 over its proxy — at least one of those two calls is redundant.
- **TM6SF2 shows the prior/floor fix is load-bearing.** All four methods find it; fb_prefix returns 7e-13 at rank 6740 of 6742 and upstream BEATRICE 1e-13 at rank 1384. The same variant, the same data, the fix alone.
- **SORT1 is where the ceiling bites.** Four variants sit at $r^2 \geq 0.99$, so the likelihood alone cannot put more than about 0.25 on any one of them; anything above that is the prior talking. funmap reports **1.000**.
- **LDLR is a correct collective miss.** Eight variants at $r^2 \geq 0.99$ (39 at ≥ 0.95), exactly as Fairoozy et al. describe. fb_xregion ranks it 9th and beatrice 8th — right at the edge of the indistinguishable set, which is the right behaviour. A high PIP here would be a fault, not a success.

A funmap pathology falls out of the two non-lead rows. rs7412 carries |Z| = 82.85 and funmap ranks it **5147 of 7317**; rs6511720 carries |Z| = 44.43 and funmap ranks it **7936 of 10634**. Explaining-away is expected of any multi-SNP model, but funmap drives near-perfect partners of its chosen variant to the bottom of the block, so its PIP ordering cannot be read as an ordering of evidence. fb_xregion's corresponding ranks are 5 and 9.

Upstream BEATRICE's single confident call in the entire genome is rs7412 (0.974, rank 1). That is a validated causal variant — and it is one call across 254 blocks, in the strongest locus in the genome, from a column whose top variant carries |Z| < 2 in 153 of 185 signal regions (§6.4). Report it as a coincidence of the extreme locus, not as a capability.

6.12 Coding-consequence enrichment: a check that needs no truth set

Seven controls cannot separate methods. This one scales to every call and needs no known causals: of the variants a method declares causal at PIP ≥ 0.95, what share are protein-altering? The reference is the set of GWAS lead variants — what you would pick with no fine-mapping at all. Consequences come from the Ensembl VEP REST endpoint queried **by rsID**, so no GRCh37/38 liftover enters; cached in vep_consequences.json, reproduced by 13_coding_enrichment.py. All 1,040 ids resolved.

set	calls	protein-altering	%	95% CI	× lead rate
polyfun_combined	69	23	**33.3%**	[23.4, 45.1]	4.4×
polyfun_sniff	83	25	**30.1%**	[21.3, 40.7]	4.0×
fb_xregion	136	23	16.9%	[11.5, 24.1]	2.2×
funmap	263	33	12.5%	[9.1, 17.1]	1.7×

set	calls	protein-altering	%	95% CI	× lead rate
fb_prefix	653	14	**2.1%**	[1.3, 3.6]	**0.3×**
GWAS lead variants	185	14	7.6%	[4.6, 12.3]	1.0×

On this metric PolyFun wins, clearly, and `fb_xregion` is second. PolyFun recovers as many protein-altering variants as `fb_xregion` (23) from half the calls (69 against 136). Report it that way; it is the one headline measure here that does not favour our own method.

`fb_prefix` is DEPLETED relative to doing no fine-mapping at all — 2.1% against the leads' 7.6%. Its 653 calls are not merely unselective, they point away from coding variants, which is independent confirmation that the saturated posterior is noise rather than a different answer.

Three limits travel with these numbers:

1. **Protein-altering consequence is a proxy for causality**, and a biased one: it counts a genuinely regulatory causal variant as a failure. SORT1 `rs12740374` is the canonical lipid example, and it is non-coding. A method that resolves regulatory variants well is penalised here.
2. **It is partly circular for a functionally informed method.** PolyFun's `combined` prior is built on baselineLF, which carries explicit coding and conserved categories, so that column is partly instructed to upweight coding variants. `polyfun_sniff` is not — it sees only the Borzoi PCs, the same information `fb_xregion`'s head sees — and still reaches 30.1% against 16.9%. That pair is the comparison worth reading, and it does not go our way.
3. **The sets overlap**, each other and the lead set: 39–43% of each method's calls are its block's lead. The intervals are Wilson binomial intervals for each set on its own; no test between sets is offered, because they are not independent samples. `polyfun_combined` [23.4, 45.1] and `fb_xregion` [11.5, 24.1] barely separate.

6.13 What this does and does not establish

Does. Both fixes to our own code are necessary and they do different jobs (§6.5). `fb_xregion` leads on localisation (134/185), has the smallest credible sets among the confident methods (median 2), places one call on a null against funmap's 61, and is the only method of the four to recover APOE ε2 (§6.11). **PolyFun, not `fb_xregion`, leads on coding enrichment** (§6.12), so the two families win different measures and the honest summary names both. The PolyFun priors run on PolyFun's documented default bin selection (30 bins, chosen automatically); the earlier 10-bin pinning was a diagnostic that `--nnls-exact` made unnecessary.

Does not. There is no ground truth here, so nothing above is power, FDR or calibration. The `beatrice` → `fb_memomean` rung confounds four changes. Roughly half the stage-1 per-SNP heritabilities are negative and PolyFun's clamp truncates them (§1.5), so both PolyFun columns rest on heavily truncated priors. 61 blocks carry a non-trivial LD distortion from the adaptive floor and the cause of the poor conditioning is unresolved. And `fb_xregion` still places **12 confident calls in the 69 blocks with no genome-wide signal** (funmap 25, PolyFun 2–3; `fb_prefix` 218): sub-threshold loci can be real, but PolyFun's 2 is the contrast that needs an answer.

---

7. Open items

- **Pre-registered prediction — ANSWERED, as a miss.** `RNAGTEX-PC5` survives LassoNet selection but ranks 10th of the 14 surviving annotations, 62× below `RNA-PC12` (§6.10). Report it as a miss; do not restate it as a confirmation on the grounds that it survived.
- **Correctness anchor — still open.** Funmap's paper fine-maps cholesterol at chr8:5–8 Mb using this same LD bucket. Reproducing that locus would validate the whole chain end to end. Note that `ldEUR_08_0011/0012/0013/0015` are excluded by the unannotated-gap rule, and `ldEUR_08_0012` carries a |Z| = 18.05 lead, so say so explicitly rather than reporting a block count.

- **Why the LD is poorly conditioned.** Four explanations tested and refuted (§6.9); pairwise-complete estimation in the published UKBB panel is the remaining untested candidate. Unresolved, and written up as unresolved.
- **Capped-floor sensitivity.** The adaptive floor runs from 0.0171 to 0.6311. `min(max(1e-3, |min eig|), 0.05)` would give the low end more protection and stop the tail over-regularising. ~25 h for the `03` re-run; not yet done.
- **Confident calls where there is no signal.** `fb_xregion` makes 11 in the 69 blocks without a genome-wide signal, against PolyFun's 2–3 (§6.12).
- **Scope beyond this application.** `prior_location` was `''` on every code path before `fix/prior-passthrough-and-ld-floor`, so every functional BEATRICE number in the LSR and the Bioinformatics paper was computed under a **uniform** prior. Deciding which of those to re-run is a separate call.
- **Merge `fix/prior-passthrough-and-ld-floor`** into functional BEATRICE's `main`; it is unmerged at `f3bd29a`, which is what `setup.sh` pins.

Appendix: figures

Produced by `12_figures.py` from the export bundle, not from the cluster. Colour marks the four methods; grey marks the three ablations of our own code, which are states, not methods (§6.1).

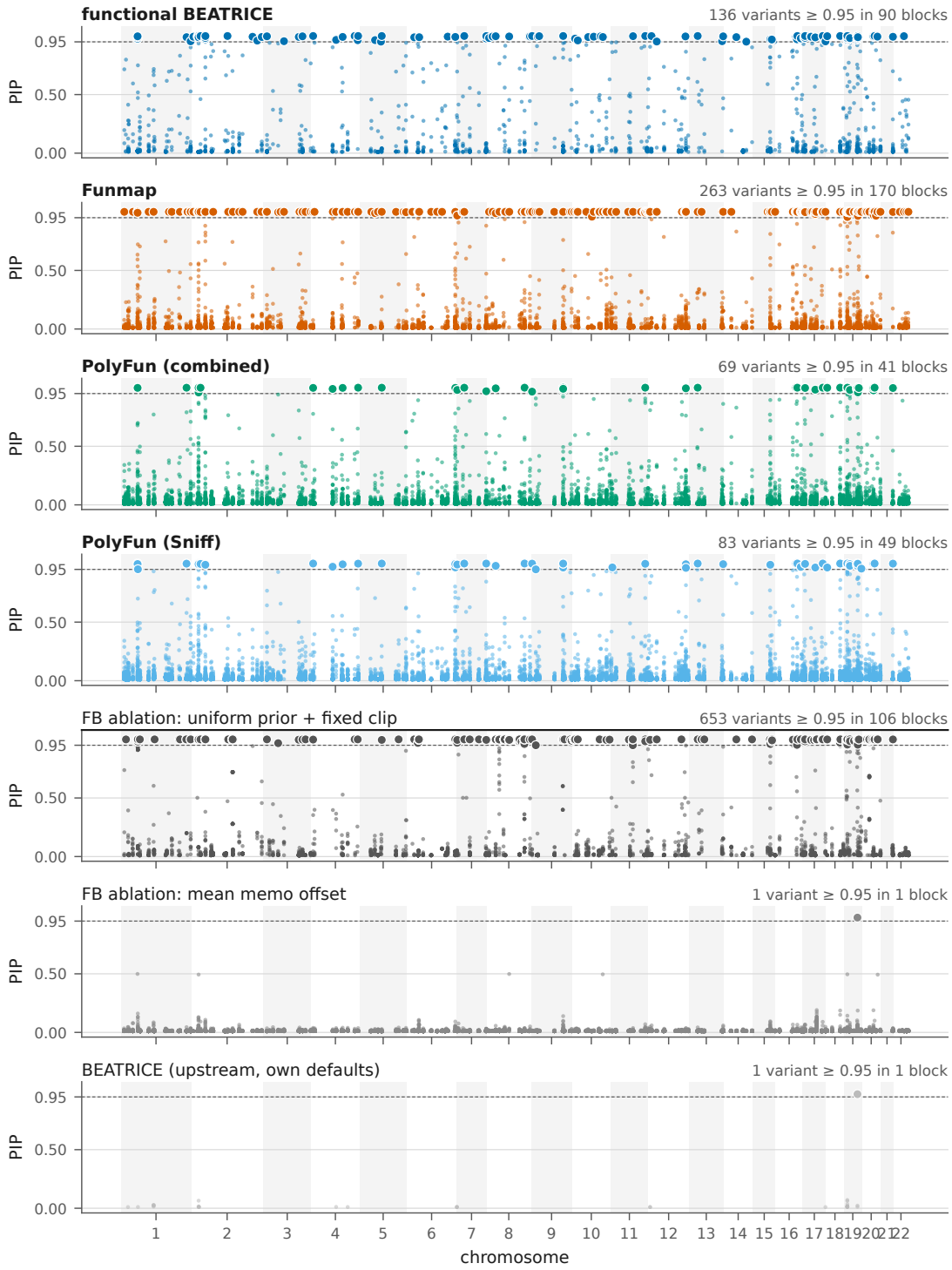


Figure 1: Posterior inclusion probabilities across all 254 LDetect blocks, UK Biobank total cholesterol. Four methods above the rule, three ablations below it. Each panel's right-hand note counts the variants that panel calls at PIP ≥ 0.95 and the blocks holding them (§6.3).

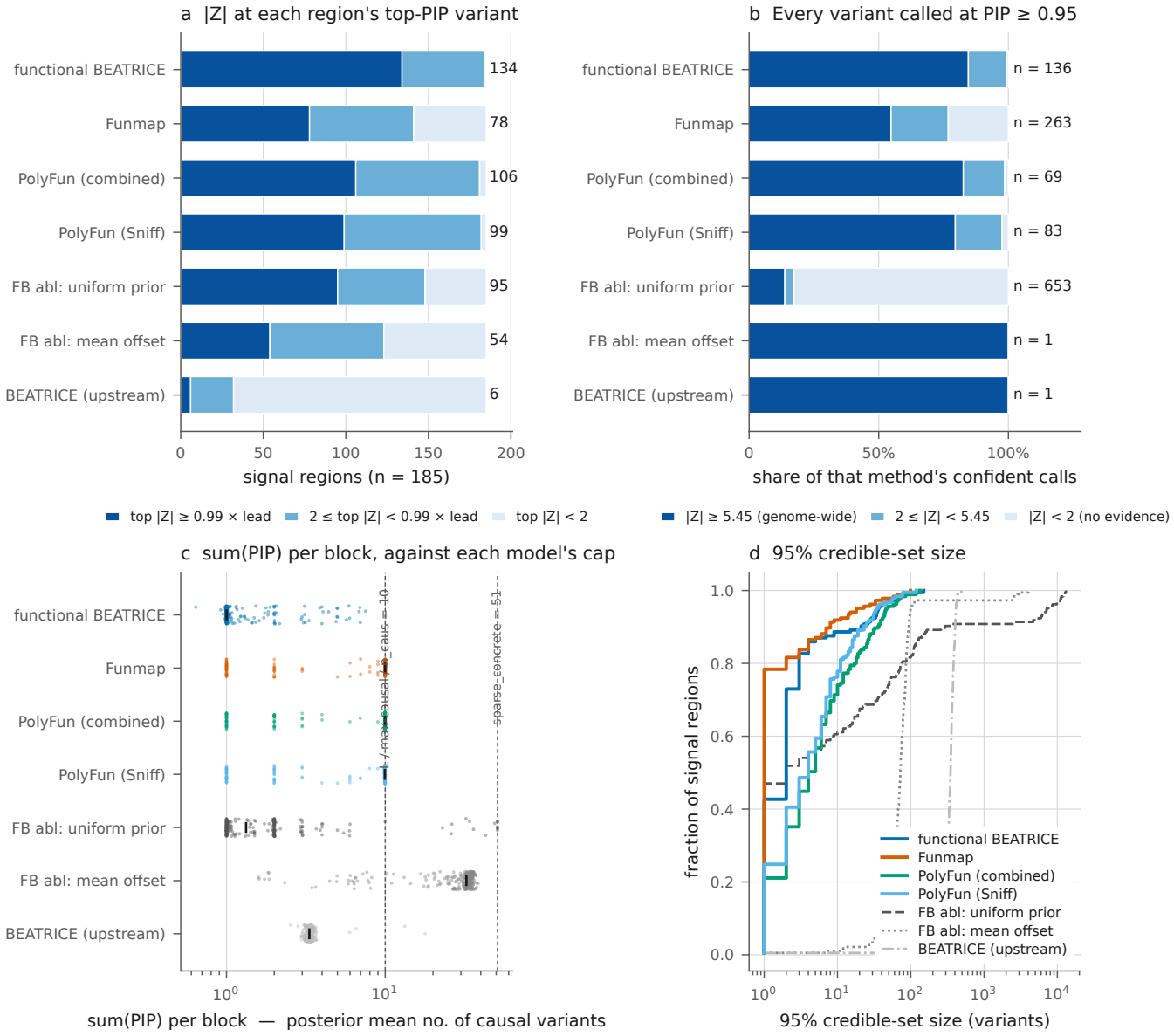
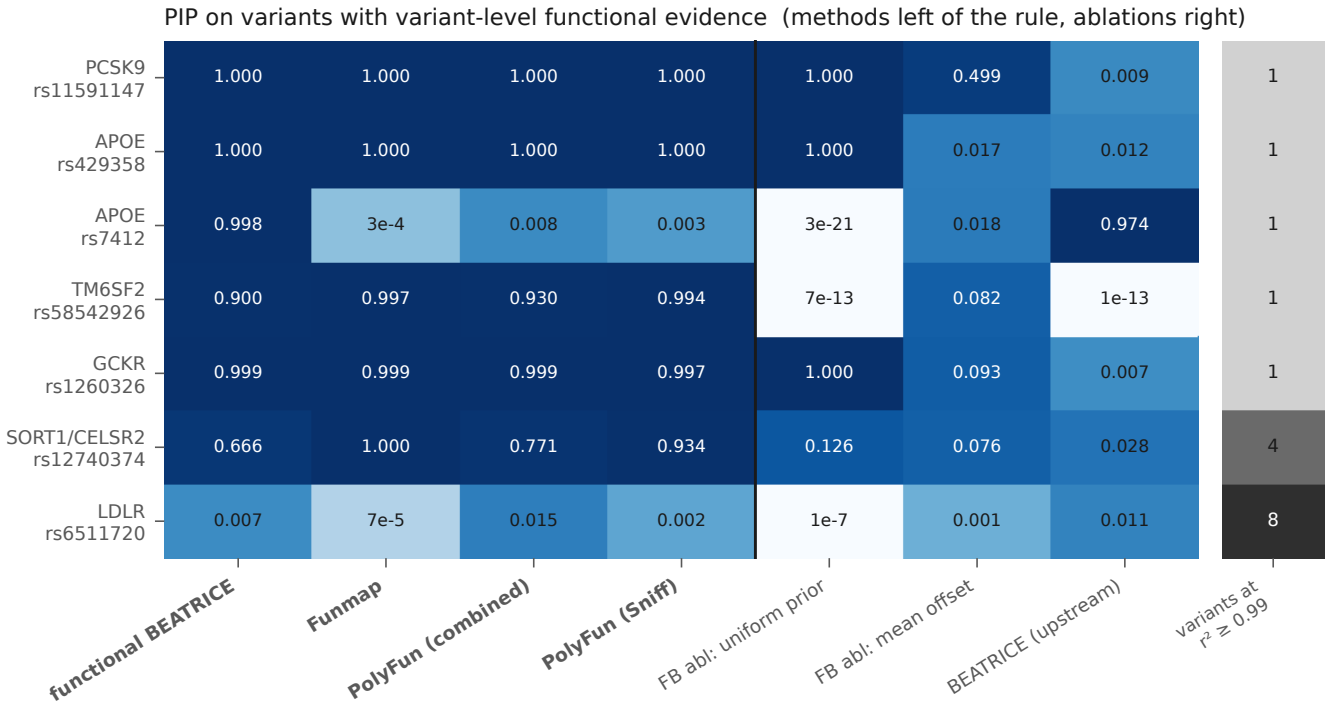


Figure 2: Method behaviour. (a) the |Z| carried by each region's own top-PIP variant (§6.4); (b) every variant called at PIP ≥ 0.95, against its |Z| (§6.3); (c) sum(PIP) per block against each model's own cap, the saturation check (§6.6); (d) 95% credible-set size.



Where k variants sit at $r^2 \geq 0.99$, association data alone cannot exceed about 1/k on any one of them — SORT1 k = 4, LDLR k = 8. A PIP near 1 there is carried by the prior, not the likelihood. rs7412 and rs6511720 are not their block's lead variant; the other five are.

Figure 3: The seven positive controls (§6.11). Methods left of the rule, ablations right. The last column gives k, the number of variants at $r^2 \geq 0.99$ with the control: where $k > 1$ the association data alone cannot put more than about $1/k$ on any one of them, so a PIP near 1 there is the prior talking.